

lab6

March 5, 2026

By: Rafay Siddiqui

Student No: 24K-0009

Section: BAI-4A

Date: 5.3.2026

1 Task 1

Write a Python program to implement the Beam Search algorithm to find the lowest-cost path from a start node to a goal node in a given graph. The algorithm should keep only the top k best paths at each level based on cumulative cost. Test the algorithm with different beam widths (k = 1, 2, 3) and display the path and total cost found in each case.

S -> A (2)

S -> B (5)

S -> C (4)

A -> D (7)

A -> E (3)

B -> F (6)

C -> G (2)

D -> T (4)

E -> T (6)

F -> T (5)

G -> T (3)

```
[31]: graph = {  
    'S': {('A',2), ('B',5), ('C',4)},  
    'A': {('D',7), ('E',3)},  
    'B': {('F',6)},  
    'C': {('G',2)},  
    'D': {('T',4)},  
    'E': {('T',6)},  
    'F': {('T',5)},  
    'G': {('T',3)},  
}
```

```

"""
1. move to layer 1, choose k cheapest nodes,
2. expand all the k nodes, aka get all neighbors, choose all k best nodes from
   ↳ this new layer
3. repeat until goal found,
"""

def beam_search(graph, start, goal, k):
    # Each path is stored as (total_cost, [list_of_nodes])
    beam = [(0, [start])]

    while beam:
        all_candidates = []

        # Expand all paths in the current beam
        for cost, path in beam:
            last_node = path[-1]

            # If we reached the goal, this path is a candidate
            if last_node == goal:
                all_candidates.append((cost, path))
                continue

            # Explore neighbors
            for neighbor, weight in graph.get(last_node, []):
                new_path = path + [neighbor]
                new_cost = cost + weight
                all_candidates.append((new_cost, new_path))

            # Sort all candidates by cost and pick the top k
            all_candidates.sort(key=lambda x: x[0])
            beam = all_candidates[:k]

            # Check if the best path in our beam has reached the goal
            # (Optional: stop early if the top path is the goal)
            if beam and beam[0][1][-1] == goal:
                return beam[0]

    return None, None

for k in [1, 2, 3]:
    cost, path = beam_search(graph, 'S', 'T', k)
    print(f"k={k}: Cost = {cost}, Path = {' -> '.join(path)}")

```

```

# for i in graph:
#     print(i,end=' ')
#     # print(graph[i])
#     for node,cost in graph[i]:
#         print(node,'-',cost,end='. ')
#     print()

# print(graph['S'])

```

k=1: Cost = 11, Path = S -> A -> E -> T
k=2: Cost = 9, Path = S -> C -> G -> T
k=3: Cost = 9, Path = S -> C -> G -> T

2 Task 2

Implement the Hill Climbing algorithm to maximize the function

$$f(x) = -x^2 + 10x + 5$$

where $0 \leq x \leq 100$

- Start from a random value of x, check neighbors (x-1 and x+1), and move to a better neighbor until no improvement is possible. Run it multiple times.

```

[35]: import random

def f(x):
    # Using the quadratic version to create a mathematical "hill"
    return -(x**2) + 10 * x + 5

def hill_climbing():
    current_x = random.randint(0, 100)

    while True:
        current_val = f(current_x)

        neighbors = []
        if current_x > 0:
            neighbors.append(current_x - 1)
        if current_x < 100:
            neighbors.append(current_x + 1)

        best_neighbor = None
        best_val = current_val

        for neighbor in neighbors:
            val = f(neighbor)
            if val > best_val:

```

```

        best_val = val
        best_neighbor = neighbor

    if best_neighbor is None:
        return current_x, current_val

    current_x = best_neighbor

print("Running Hill Climbing 5 times:")
for i in range(1, 6):
    best_x, max_value = hill_climbing()
    print(f"Run {i}: Peak found at x = {best_x}, f(x) = {max_value}")

```

```

Running Hill Climbing 5 times:
Run 1: Peak found at x = 5, f(x) = 30
Run 2: Peak found at x = 5, f(x) = 30
Run 3: Peak found at x = 5, f(x) = 30
Run 4: Peak found at x = 5, f(x) = 30
Run 5: Peak found at x = 5, f(x) = 30

```

3 Task 3:

Implement a Genetic Algorithm to minimize assignment cost for 10 tasks and 5 machines using this 10×5 cost matrix:

```

cost_matrix = [
    [4, 6, 8, 7, 5],
    [7, 5, 6, 8, 4],
    [6, 4, 7, 5, 8],
    [5, 8, 6, 4, 7],
    [8, 6, 5, 7, 4],
    [7, 4, 8, 6, 5],
    [6, 7, 4, 5, 8],
    [5, 6, 7, 8, 4],
    [4, 7, 5, 6, 8],
    [8, 5, 6, 4, 7]
]

```

Each chromosome is a list of length 10, where each index represents a task and the value is the assigned machine. Include random population initialization, fitness = 1/total cost, top-50% selection, crossover, mutation, and 100 generations. Print the best chromosome, total cost, fitness, and generation. Run with population sizes 10 and 30 and compare results.

```
[37]: import random
```

```

cost_matrix = [
    [4, 6, 8, 7, 5],
    [7, 5, 6, 8, 4],

```

```

[6, 4, 7, 5, 8],
[5, 8, 6, 4, 7],
[8, 6, 5, 7, 4],
[7, 4, 8, 6, 5],
[6, 7, 4, 5, 8],
[5, 6, 7, 8, 4],
[4, 7, 5, 6, 8],
[8, 5, 6, 4, 7]
]

def run_ga(pop_size):
    # create initial random population
    population = []
    for _ in range(pop_size):
        chrom = [random.randint(0, 4) for _ in range(10)]
        population.append(chrom)

    best_chrom = None
    best_cost = 9999 # start with a super high dummy cost
    best_gen = 0

    # run for 100 generations
    for gen in range(100):
        pop_scores = []

        # 1. calculate cost and fitness for everyone
        for chrom in population:
            cost = 0
            for task in range(10):
                machine = chrom[task]
                cost += cost_matrix[task][machine]

            fitness = 1 / cost
            pop_scores.append((cost, fitness, chrom))

        # save the all-time best
        if cost < best_cost:
            best_cost = cost
            best_chrom = list(chrom)
            best_gen = gen

        # 2. sort by cost (lowest cost comes first)
        pop_scores.sort()

        # 3. selection (keep the top 50%)
        survivors = []
        for i in range(pop_size // 2):

```

```

        survivors.append(pop_scores[i][2])

    # 4. crossover (mix parents to fill the rest of the population)
    next_gen = list(survivors)
    while len(next_gen) < pop_size:
        p1 = random.choice(survivors)
        p2 = random.choice(survivors)
        cut = random.randint(1, 8)

        # combine first part of p1 with second part of p2
        child = p1[:cut] + p2[cut:]
        next_gen.append(child)

    # 5. mutation (randomly change a machine assignment)
    # start at index 1 so we don't mutate our best survivor
    for i in range(1, pop_size):
        for j in range(10):
            if random.random() < 0.1: # 10% chance to mutate
                next_gen[i][j] = random.randint(0, 4)

    population = next_gen

    # print final results
    print("Best Chromosome:", best_chrom)
    print("Total Cost:", best_cost)
    print("Fitness:", round(1 / best_cost, 4))
    print("Found in Gen:", best_gen)
    print("-" * 30)

print("Running GA with Population 10:")
run_ga(10)

print("Running GA with Population 30:")
run_ga(30)

```

```

Running GA with Population 10:
Best Chromosome: [0, 4, 1, 3, 4, 1, 2, 4, 0, 3]
Total Cost: 40
Fitness: 0.025
Found in Gen: 35

```

```

-----
Running GA with Population 30:
Best Chromosome: [0, 4, 1, 3, 4, 1, 2, 4, 0, 3]
Total Cost: 40
Fitness: 0.025
Found in Gen: 33

```